

Search "does not work in any scenario" — consolidated 5-layer root-cause (2026-06-14)

Status: Layers 1 + 2 + 3 fixed (L1+L2 on-device-verified; L3 RED→GREEN at the test layer). Layer 4 (READ_API_KEY permission grant) proven to be a **TEST-VM ARTIFACT, production-safe**. Layer 5 (Lava-Auth key not reaching the search request) is now **ROOT-CAUSE-FOUND + FIX-APPLIED (on-device verification in progress)**: the api-app's ApiKeyProvider ContentProvider cached its key/port resolver lambdas in onCreate() gated on the ApiApplication holders being non-null, on a FALSE assumption about Android's lifecycle ordering (ContentProvider.onCreate runs BEFORE Application.onCreate, not after) — so the holders were always null at cache time and the provider served an EMPTY cursor for the whole process, dropping the key. The fix resolves the holders LAZILY per query(). **The fix is applied and the root cause is understood; the on-device proof that search returns RESULTS is still running** — so search is **NOT yet claimed to work on-device** and 1.3.9 is **not shipped** until that device gate is GREEN.

Release scope change: 1.3.9 is **NO LONGER client-only** — the L5 fix is in the **api-app**, so **client 1.3.9-1066 + api-app 0.2.9-14 ship together** (\$6.Y bump applied).

Fix commits: d05bc71e (L1+L2), 21031f2a (L3 key restore), addaacd0 (L4 cleanup — dropped ignored authority param + dead SSE path), plus the L5 api-app ApiKeyProvider lazy-resolution fix (this cycle). **Operator report:** "search still does not work in any scenario" (1.3.8-1065).

No-guessing note (§11.4.6): every claim below is anchored to git truth (the fix diffs + the incident JSON `.lava-ci-evidence/sixth-law-incidents/2026-06-14-sse-search-v1search-unserved-bluff.json`

- the L4 analysis docs/issues/2026-06-14-readapikey-permission-variant-analysis.md
- the L5 on-device pinpoint `.lava-ci-evidence/search-verification/2026-06-14-keyloss-pinpoint.md`) or marked VERIFYING / OWED. The L5 root cause and fix are asserted as fact (anchored to the source + the LAVAKEYDBG logcat pinpoint); the on-device proof that **search returns results** is VERIFYING

(in progress) — search is **NOT claimed to work on-device** until that gate is GREEN.

1. The cascade (five independent bugs, one symptom)

A single user-visible failure ("search returns nothing / error in any scenario") was the surface of FIVE independent defects stacked on the GoApi search path. Each had to be addressed; fixing only some still leaves search broken. The cascade was peeled back one layer at a time from on-device evidence across this cycle: the first three were client-side code defects, the fourth a test-environment artifact, and the fifth (root-caused + fixed this cycle, on-device verification in progress) is an **api-app** defect — the per-request auth key was being dropped at its source (the key ContentProvider served an empty cursor for the whole process) so it never reached the search call.

Layer 1 — onboarding never wrote settings.endpoint

OnboardingViewModel.onSelectApi persisted the chosen API to:

- the Room Endpoint list (endpointsRepository.add(endpoint)), and
- ApiBaseUrlHolder (the dynamic-SDK / GET /providers seam),

but **never** to settings.endpoint. The home search resolves its target host from SettingsRepository.getSettings().endpoint via NetworkApiRepositoryImpl.endpoint() — a DIFFERENT store from the two above. It stayed at the default Endpoint.GoApi("lava-api.local"), which does not resolve on the LAN → UnknownHostException. So /providers worked (it reads ApiBaseUrlHolder) while search died (it reads settings.endpoint).

Fix: onSelectApi now invokes SetEndpointUseCase(endpoint) (nullable injected seam, real impl bound by Hilt; null default preserves direct-construction tests). RepopulateProvidersOnStartupUseCase.reconcileActiveEndpoint() heals existing installs whose onboarding pre-dated this fix — when settings.endpoint is the orphan default, it adopts the persisted Room GoApi.

On-device proof: post-fix, search reaches 10.0.3.16, no longer lava-api.local.

Layer 2 — GoApi search routed to GET /v1/search, a route NO backend serves

SearchResultViewModel's multi-provider dispatch sent every GoApi search to observeSseSearch → GET {base}/v1/search. That route is registered by **no backend** (see §3). Result: 404 for every GoApi user, regardless of host or auth.

Fix: route GoApi multi-provider search to observeStreamMultiSearch → sdk.streamMultiSearch → GET /v1/{provider}/search (the served path; carries the per-instance Lava-Auth key + the permissive-LAN OkHttp client wired in 1.3.8 via ApiBaseUrlHolder). The dispatch change appears in both searchClicked-class entry points in SearchResultViewModel.

Layer 3 — mDNS-discovered endpoint persisted without its Lava-Auth key (FIXED)

After Layers 1+2, the endpoint that onboarding persisted for an mDNS-discovered API carried **no per-instance key**: onSelectApi adopted the discovery record (host/port/platform) but never attached the locally-readable Lava-Auth key, so the persisted settings.endpoint was keyless → every /v1 op 401'd even when the key was readable on the device.

Fix (21031f2a): onSelectApi now reads the local api-app key (via the variant-aware key reader) for keyless mDNS-discovered endpoints and persists the GoApi WITH its key; an app-side cold-start key-restore path heals an already persisted keyless endpoint by re-reading the local key. Both the onboarding-time and the startup-restore paths were RED→GREEN at the test layer (the reproduction asserts the persisted endpoint carries the key; without the read, the endpoint stays keyless).

Layer 4 — READ_API_KEY permission not granted on the test VM (TEST-VM ARTIFACT, production-safe)

After Layer 3, /v1/{provider}/search STILL 401'd on the Genymotion VM because the variant-aware key reader returned null — the signature permission digital.vasic.lava.permission.READ_API_KEY that guards the api-app's key ContentProvider was **not granted** to the client on the VM.

Root cause (proven, not guessed — full evidence in docs/issues/2026-06-14-readapikey-permission-variant-analysis.md): the permission name is a **fixed literal** (no .dev suffix, no placeholder) in both manifests, while every other cross-app identifier (applicationId, provider authority \${apiKeyAuthority}, API_TARGET_PACKAGE) IS variant-suffixed. The VM had **both**

differently-signed api-app variants in its install history (debug ... api.dev from debug.keystore AND release ...api from release.keystore). Two differently-signed packages contending to DEFINE the same fixed-name signature permission is the classic Android INSTALL_FAILED_DUPLICATE_PERMISSION trap: the client signed with the OTHER cert than the current definition-owner is denied the grant → reader returns null → 401.

Verdict: production is SAFE. The shipped pair (release client + release api-app, both release.keystore, the only combination end users install) grants READ_API_KEY at install — certs match. **A clean-install matched-pair run confirmed this on-device: granted=true.** The observed 401 was the test VM's mixed debug/release co-install, which real users never reproduce.

Recommended hardening (MEDIUM priority, QA-fidelity not production-correctness): variant-suffix the permission name (same pattern as the authority) so debug and release never define the same name. This makes the test VM exercise the same grant path production hits (\$6.J — a test environment that can't grant the permission is testing a different thing than production). Not shipping-blocking; production already grants it.

Layer 5 — api-app ApiKeyProvider served an empty cursor for the whole process (ROOT-CAUSE-FOUND + FIX-APPLIED; device-verify in progress)

This was the **load-bearing layer** and is now **root-caused and fixed in code**; the on-device proof that search returns RESULTS is still running. On a CLEAN matched pair — release client + release api-app, same signature, granted=true, engine running, fresh onboard — /v1/{provider}/search STILL returned 401 because the per-instance Lava-Auth key was being dropped **at its source on the api-app side** before the client could ever read it.

Root cause (proven, not guessed): api-app/src/main/kotlin/lava/api/app/handoff/ApiKeyProvider.kt cached its key and port resolver lambdas in ContentProvider.onCreate(), gated on ApiApplication.controllerHolder / keyStoreHolder being non-null, on the FALSE inline comment "ContentProvider.onCreate runs AFTER Application.onCreate." Android runs ContentProvider.onCreate **BEFORE** Application.onCreate. The holders are set inside ApiApplication.onCreate (lines 46-49), so at ApiKeyProvider.onCreate time they were **always null** → the resolver lambdas stayed { null } for the entire process lifetime → every query() served an **empty cursor**. The client's ApiKeyClient.read() therefore got null → withLocalApiKeyIfMissing produced a keyless endpoint → ApiBaseUrlHolder.set(url, null) → ApiBackedTrackerClient was built with authKey=null → no Lava-Auth

header → every /v1/{provider}/search 401'd. Public routes (/providers, /health) need no key so they worked — which is exactly why search NEVER worked while everything else did.

On-device pinpoint evidence: .lava-ci-evidence/search-verification/2026-06-14-keyloss-pinpoint.md — the LAVAKEYDBG logcat line with LocalApiKey readerSet=true readLen=-1 shows the key read returned null **at the source** (reader wired, read length -1), pinning the drop to the ContentProvider, not the client.

The withFakes bluff: the existing ApiKeyProvider unit test exercised the provider through a test-only withFakes seam that injected the resolver lambdas directly, **bypassing the real onCreate → holder path** — i.e. it skipped the exact lifecycle ordering where the bug lives. It passed green while the production path served an empty cursor. A real-holder regression test (driving the real onCreate ordering) is being added so this class of defect cannot recur green.

Fix (applied): ApiKeyProvider no longer caches resolvers in onCreate(). It resolves the holders + the engine's Running state **LAZILY per query()** (via resolveRunningKey() / resolveRunningPort() defaults), removing the onCreate-ordering dependency entirely. By the time any query() arrives the client process has long since finished ApiApplication.onCreate, so the holders are populated and the real key flows.

Important evidence caveat (unchanged): a public GET /providers → 200 on the same host proves only **reachability + that the host/TLS path works** — /providers is registered BEFORE the auth middleware (the 2026-06-13 onboarding fix), so a 200 there does NOT prove the Lava-Auth key is present or valid. Only an authed /v1/{provider}/... request exercises the key.

Status: the fix is applied and the root cause is understood. The on-device proof that search returns RESULTS (authed /v1/{provider}/search → 200 + result rows) is **VERIFYING (in progress)**. Until that run is GREEN, **search is not claimed to work on-device** and the bundled release (client 1.3.9-1066 + api-app 0.2.9-14) does not ship. No layer above this one makes search functional on its own.

2. The two search dispatch paths

SearchResultViewModel chooses a search transport per the current filter + endpoint:

Path	Transport	HTTP route	Status
observePagingData	paging (legacy)	GET /search	served (legacy/standalone)
observeSseSearch	SSE consumer	GET /v1/search	REMOVED (addaacd0) — was DEAD, no backend served it
observeStreamMultiSearch	sdk.streamMultiSearch (client-side fan-out)	GET /v1/{provider}/search	served by both backends — the working path

Before the fix, GoApi multi-provider search took the **dead** `observeSseSearch` → `/v1/search` branch. After the fix it takes `observeStreamMultiSearch` → `/v1/{provider}/search`, fanning out per provider client-side. The dead `observeSseSearch` → `/v1/search` consumer was **deleted entirely** in `addaacd0` (no backend ever served `/v1/search`), so the dispatch can no longer regress onto it.

3. Served-route facts (verified across the Go codebase)

- `internal/router/router.go` (embedded api-app + shared via `internal/mobile`): serves `/providers`, `/v1:provider/{op}`, `/jackett/search`, `/health/ready`.
- The standalone `lava-api-go`: serves legacy `/search` and `/v1/{provider}/search`.
- **Neither registers `/v1/search`.** That is why the (now-removed) `observeSseSearch` 404'd for every GoApi user.

4. The endpoint-store split (why Layer 1 was invisible)

There are **three** distinct places "the active API" is recorded, and the search path reads a different one than onboarding wrote:

1. **Room Endpoint table** — the user's saved endpoint list (`endpointsRepository`). Holds `host/platform/storage`; the per-instance auth key is NOT a column here.

2. **settings.endpoint** — a single serialized active Endpoint inside settings, stored/decoded via EndpointConverter. This serialized form **carries the per-instance Lava-Auth key**. NetworkApiRepositoryImpl.endpoint() (the search/network host resolver) reads THIS.
3. **ApiBaseUrlHolder** — an in-memory active-base-URL + key holder consumed by the dynamic SDK clients (ApiBackedTrackerClient) + GET /providers.

Onboarding wrote (1) and (3) but not (2) → search read a stale (2). The key-bearing nature of (2) via EndpointConverter is also why an existing-install heal that adopts the keyless Room row (1) yields a 401 (see §6).

5. The key-flow (how the per-instance Lava-Auth key reaches a request)

ApiKeyClient (variant-aware ContentProvider reader) → goApi.key → setEndpoint persists the GoApi WITH its key into settings.endpoint (via EndpointConverter) AND ApiBaseUrlHolder.set(url, key). The GoApi branch of NetworkApiRepositoryImpl then turns host:port + key into a permissive-LAN client + withKeyOverride request, so /v1/{provider}/search SHOULD carry Lava-Auth: <key>. The key lives in (2) and (3) above — never in the Room row (1). When the ContentProvider read returns null (the L4 permission-not-granted condition), the key is absent and /v1 ops 401. **L5 is the case this chain does NOT yet explain:** the key IS readable (granted=true) yet /v1/{provider}/search still 401s — so the key is being dropped SOMEWHERE between a non-null read and the outgoing header. Pinpointing that link is the open L5 work.

6. Open follow-ups (OPEN / OWED / VERIFYING)

- **Layer 5 — api-app ApiKeyProvider empty-cursor (FIX-APPLIED; device-verify in progress).** Root cause found + fixed: the key ContentProvider cached its resolver lambdas in onCreate() gated on holders that are still null at that point (ContentProvider.onCreate runs BEFORE Application.onCreate) → empty cursor → keyless endpoint → /v1/{provider}/search 401. Fix: lazy per-query() holder resolution. The on-device proof that search returns RESULTS is **VERIFYING (in progress)** — the load-bearing gate for the bundled 1.3.9 release. **Release scope: client 1.3.9-1066 + api-app 0.2.9-14 ship together (§6.Y bumped).** A real-holder regression test (real onCreate ordering, no withFakes

seam) is OWED to retire the bypass-bluff. **Search is not claimed to work on-device until the device gate is GREEN.**

- **Layer 4 — variant-suffix the permission name (OWED, MEDIUM/QA-fidelity).** Production grant is already safe (§1 Layer 4); the suffix makes the test VM exercise the production grant path so a mixed-install VM can never recreate the `INSTALL_FAILED_DUPLICATE_PERMISSION` collision. Not shipping-blocking.
 - **Existing-install key-restore (OWED).** Installs that do NOT re-onboard get a keyless healed endpoint → `/v1 ops 401`, because the key lives only in `settings.endpoint/ApiBaseUrlHolder` and the old onboarding never wrote it, and the heal adopts the keyless Room row. Fresh onboard flows the key correctly; the L3 cold-start key-restore (21031f2a) addresses the keyless mDNS endpoint but a full existing-install restore is still owed.
 - **Dormant SSE path (CLOSED by addaacd0).** The dead `observeSseSearch` → `/v1/search` consumer was removed entirely (no backend served `/v1/search`). Three SSE bluff tests were removed alongside it. If a server-side `/v1/search` SSE aggregator is ever wanted, it is a fresh feature, not a revival.
-

7. Tests + evidence

- **Reproduction tests (real-stack, falsifiability-rehearsed, on the REAL path):**
 - `OnboardingViewModelDynamicProvidersTest` — selecting an API persists it as the active settings endpoint the search path reads (Layer 1).
 - `RepopulateProvidersOnStartupUseCaseTest` — cold start heals a stale orphan active endpoint to the onboarded server in the list (heal path).
- **Bluffs removed:** `SearchResultSseErrorRetryTest` (served `/v1/search` over a mock no backend serves) + the two remaining SSE consumer tests removed with the dead `observeSseSearch` path in `addaacd0`. Incident JSON: `.lava-ci-evidence/sixth-law-incidents/2026-06-14-sse-search-v1search-unserved-bluff.json`.
- **On-device Chucker evidence (L1/L2):** GET `https://10.0.3.16/providers` → 200; GET `https://lava-api.local/search?query=prince...` → `UnknownHostException` (api-app "Requests served: 0"). Surfaced by the HelixQA video-QA agent; traced via the flow-db Endpoint row + `chucker.db` transactions.
- **On-device evidence (L4):** clean matched-pair install → `READ_API_KEY` granted=true (production grant path confirmed). Analysis: `docs/issues/2026-06-14-readapikey-permission-variant-analysis.md`.
- **On-device evidence (L5, FIX-APPLIED):** clean matched pair, granted=true, engine running → `/v1/{provider}/search` was STILL 401 because the api-app `ApiKeyProvider` served an empty

cursor (LAVAKEYDBG logcat withLocalApiKey readerSet=true readLen=-1 = key read returned null at the source). Pinpoint: .lava-ci-evidence/search-verification/2026-06-14-keyloss-pinpoint.md. Fix applied (lazy per-query() holder resolution); on-device proof that search returns RESULTS is VERIFYING (in progress). Public /providers → 200 still proves reachability only, NOT key validity.

- **§6.T.4 BUGFIXES entry:** docs/issues/fixed/BUGFIXES.md — "search does not work in any scenario" (5-layer cascade, 2026-06-14).
-

8. §6.J meta-lesson — verify subagents root-cause, the main stream verifies before applying

A §6.J governance lesson surfaced this cycle and is recorded so it does not recur. **A verification subagent proposed an L4 fix and the main stream REJECTED it after reading the source.**

The rejected theory (worked example): the verify subagent proposed wiring a hardcoded ".keyprovider" authority string into the key-read path as the L4 fix. The main stream read the source before applying and found the proposal inert:

MainActivity.buildApiKeyReader returns { -> client.read()?.key } — a lambda that takes **no authority argument and ignores any authority passed**. Threading a ".keyprovider" authority into that call site changes nothing the reader uses; the read still goes through the variant-aware client unchanged. Applying the "fix" would have produced a green-looking diff that does not touch the actual grant condition (which is the signature-permission match, §1 Layer 4) — a §6.J bluff by construction (a change that claims to fix the thing it cannot affect). It would also have introduced a hardcoded literal in violation of §6.R.

The lesson: a verification subagent's job is to **root-cause and report**, not to auto-apply fixes. Subagent-proposed code changes **MUST** be read against the production source by the main stream before they land. A proposal that "looks like a fix" but does not touch the real defect path is a bluff regardless of the subagent's confidence. This is the same discipline §6.L restates: green-looking output is necessary, never sufficient — the main stream confirms the proposed change actually reaches the failing code path before it is applied.

The L5 §6.J meta-lesson — a test that bypasses the real lifecycle via a test-only seam is a bluff by construction. The ApiKeyProvider unit test drove the provider through a withFakes seam that injected the resolver lambdas directly, never going through the real onCreate → ApiApplication holder path. The L5 bug

lived in **exactly the path the seam skipped** — the `ContentProvider.onCreate-before-Application.onCreate` ordering that left the holders null and the cached resolvers `{ null }`. So the test passed green while the production provider served an empty cursor for the whole process. This is the canonical Sixth-Law clause-1 failure: the test did not traverse the production code path the user's action triggers. A seam that exists to make a unit test easier **MUST NOT** skip the lifecycle ordering or wiring where the defect can live; when it does, the test guarantees nothing about the real path. The remediation is a real-holder regression test that drives the actual `onCreate` ordering (no `withFakes`), so the empty-cursor defect would fail RED before the fix.